

SQLD 2 과목

SQL 기본 및 활용

[공식 2 과목 범위]

정의: SQL 기본, SQL 활용, 관리 구문 전체를 평가한다.

핵심 설명: SQL 기본은 SELECT, 함수, WHERE, GROUP BY/HAVING, ORDER BY, 조인 중심입니다. SQL 활용은 서브쿼리, 집합 연산자, 그룹 함수, 윈도우 함수, Top N, 계층형 질의, PIVOT/UNPIVOT, 정규 표현식입니다. 관리 구문은 DML, TCL, DDL, DCL 입니다.

HR 예시 쿼리:

```
-- 이 문서의 예시 기본 구조  
SELECT e.employee_id, e.last_name, d.department_name  
FROM HR.EMPLOYEES e  
LEFT JOIN HR.DEPARTMENTS d  
      ON e.department_id = d.department_id  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 2 과목은 40 문항 비중이므로 단순 암기보다 실행 결과 예측 연습이 중요합니다.

오답 방지: Oracle에서는 문자열 연결에 CONCAT 보다 || 연산자를 더 자주 사용하며, 상위 N 건은 LIMIT 이 아니라 FETCH FIRST 또는 ROWNUM 을 사용합니다.

1 절 관계형 데이터베이스 개요

[데이터베이스와 DBMS]

정의: 데이터베이스는 데이터를 체계적으로 저장한 공간이고, DBMS 는 이를 관리하는 소프트웨어다.

핵심 설명: 사용자는 SQL 을 통해 DBMS 에 명령을 내립니다. SQLD 에서는 데이터베이스 자체, DBMS, SQL 의 역할을 구분해야 합니다.

HR 예시 쿼리:

```
SELECT COUNT(*) AS employee_count
FROM HR.EMPLOYEES;
```

결과 해석: HR.EMPLOYEES 테이블에 저장된 사원 행 수를 조회합니다.

★ **시험 포인트:** SQL 은 데이터를 직접 다루는 언어이고, DBMS 는 SQL 을 받아 저장소를 관리합니다.

[관계형 데이터베이스]

정의: 데이터를 테이블 형태로 저장하고, 테이블 사이 관계로 연결하는 데이터베이스다.

핵심 설명: 테이블은 행과 열로 구성됩니다. 행은 실제 데이터 한 건, 열은 데이터 항목입니다. 기본키는 한 행을 구분하고, 외래키는 다른 테이블과의 관계를 표현합니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, e.department_id, d.department_name
FROM HR.EMPLOYEES e
JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id
FETCH FIRST 5 ROWS ONLY;
```

결과 해석: EMPLOYEES 의 DEPARTMENT_ID 가 DEPARTMENTS 의 DEPARTMENT_ID 와 연결됩니다.

★ **시험 포인트:** PK/FK 관계를 알아야 JOIN 조건을 올바르게 찾을 수 있습니다.

[SQL 명령어 분류]

정의: SQL 은 DML, TCL, DDL, DCL 로 나뉜다.

핵심 설명: DML 은 데이터를 조회·입력·수정·삭제하고, TCL 은 트랜잭션을 확정·취소합니다. DDL 은 객체 구조를 만들거나 바꾸고, DCL 은 권한을 부여·회수합니다.

HR 예시 쿼리:

-- DML 예시

```
SELECT employee_id, last_name  
FROM HR.EMPLOYEES  
WHERE employee_id = 100;
```

-- DDL 예시

```
CREATE TABLE emp_sample AS  
SELECT employee_id, last_name, salary  
FROM HR.EMPLOYEES  
WHERE 1 = 0;
```

-- DCL 예시

```
GRANT SELECT ON HR.EMPLOYEES TO user1;
```

★ **시험 포인트:** SELECT 는 DML, COMMIT 은 TCL, CREATE 는 DDL, GRANT 는 DCL 입니다.

오답 방지: DELETE 는 DML 이지만 TRUNCATE 는 DDL 성격으로 분류하는 문제가 자주 나옵니다.

[관계 연산]

정의: 선택, 투영, 조인으로 원하는 결과를 만든다.

핵심 설명: 선택은 행을 고르는 연산, 투영은 열을 고르는 연산, 조인은 여러 테이블을 연결하는 연산입니다.

WHERE 는 선택, SELECT 목록은 투영, JOIN 은 연결로 이해하면 됩니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name -- 투영  
FROM HR.EMPLOYEES e  
JOIN HR.DEPARTMENTS d -- 조인  
  ON e.department_id = d.department_id  
WHERE e.salary >= 10000; -- 선택
```

★ **시험 포인트:** SQL 문장을 관계 연산 관점으로 읽으면 SELECT 구조가 쉬워집니다.

2 절 SELECT 문

[SELECT 문]

정의: 테이블에서 필요한 데이터를 조회하는 SQL 문이다.

핵심 설명: SELECT 는 어떤 컬럼을 보여줄지, 어느 테이블에서 가져올지, 어떤 조건을 적용할지 지정합니다.

시험에서는 문법 순서와 실제 논리 처리 순서를 구분해야 합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE department_id = 60
ORDER BY salary DESC;
```

★ **시험 포인트:** SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY 의 역할을 구분해야 합니다.

[SELECT 문법 순서와 논리적 처리 순서]

정의: SQL 을 쓰는 순서와 DBMS 가 논리적으로 처리하는 순서는 다르다.

핵심 설명: 문법은 SELECT 부터 쓰지만 논리적으로는 FROM 에서 대상 테이블을 정하고, WHERE 로 행을 거른 뒤 GROUP BY 와 HAVING 을 처리하고, SELECT 목록을 계산한 후 ORDER BY 로 정렬합니다.

HR 예시 쿼리:

```
SELECT department_id, AVG(salary) AS avg_salary
FROM HR.EMPLOYEES
WHERE salary IS NOT NULL
GROUP BY department_id
HAVING AVG(salary) >= 7000
ORDER BY avg_salary DESC;
```

결과 해석: WHERE 가 먼저 개별 행을 거르고, GROUP BY/HAVING 이 그룹을 처리한 뒤, ORDER BY 가 별칭 AVG_SALARY 를 사용할 수 있습니다.

★ **시험 포인트:** 논리적 순서: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> FETCH

오답 방지: WHERE 절에서는 SELECT 별칭 AVG_SALARY 를 일반적으로 사용할 수 없습니다.

[DISTINCT]

정의: SELECT 결과에서 중복 행을 제거한다.

핵심 설명: DISTINCT 는 선택한 컬럼 조합 전체가 같은 행을 중복으로 봅니다. 특정 컬럼 하나만 따로 중복 제거하는 것이 아니라 SELECT 목록 전체 기준입니다.

HR 예시 쿼리:

```
SELECT DISTINCT department_id, job_id
FROM HR.EMPLOYEES
ORDER BY department_id, job_id;
```

★ **시험 포인트:** DISTINCT 는 원본 데이터를 삭제하지 않고 결과 집합에서만 중복을 제거합니다.

오답 방지: DISTINCT department_id, job_id 는 department_id 만 중복 제거하는 것이 아니라 두 컬럼 조합 기준입니다.

[별칭(Alias)]

정의: 컬럼이나 테이블에 임시 이름을 붙이는 것이다.

핵심 설명: Oracle 에서는 컬럼 별칭에 AS 를 사용할 수 있지만 테이블 별칭에는 AS 를 쓰지 않습니다. ORDER BY 에서는 SELECT 별칭을 사용할 수 있습니다.

HR 예시 쿼리:

```
SELECT e.employee_id AS emp_id,
       e.last_name   AS name,
       e.salary * 12 AS annual_salary
FROM HR.EMPLOYEES e
ORDER BY annual_salary DESC
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** ORDER BY 는 SELECT 이후 처리되므로 SELECT 별칭 사용이 가능합니다.

오답 방지: Oracle 에서 FROM HR.EMPLOYEES AS e 처럼 테이블 별칭에 AS 를 쓰면 오류가 날 수 있습니다.

[산술식과 문자열 연결]

정의: SELECT 목록에서 계산식이나 문자열 연결식을 사용할 수 있다.

핵심 설명: Oracle에서는 문자열 연결에 || 연산자를 사용합니다. 산술식에 NULL이 포함되면 결과가 NULL이 될 수 있으므로 NVL 등으로 처리합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       first_name || ' ' || last_name AS full_name,  
       salary * 12 AS annual_salary,  
       salary + NVL(commission_pct * salary, 0) AS monthly_pay_with_commission  
FROM HR.EMPLOYEES  
WHERE employee_id IN (100, 145, 146);
```

★ **시험 포인트:** NULL이 산술식에 들어가면 결과가 NULL이 될 수 있습니다.

오답 방지: salary + commission_pct * salary에서 COMMISSION_PCT가 NULL이면 전체 계산 결과가 NULL이 될 수 있습니다.

[DUAL]

정의: Oracle에서 테이블 없이 표현식이나 함수를 테스트할 때 쓰는 1행짜리 더미 테이블이다.

핵심 설명: Oracle은 SELECT문에 FROM절이 필요한 경우가 많아 단순 계산이나 함수 테스트에 DUAL을 사용합니다.

HR 예시 쿼리:

```
SELECT SYSDATE AS today,  
       NVL(NULL, 0) + 10 AS result  
FROM DUAL;
```

★ **시험 포인트:** Oracle 함수 결과 예측 문제에서는 DUAL을 사용한 예시가 자주 보입니다.

3 절 함수

우선 암기 대상: NVL/NVL2/COALESCE/NULLIF, CASE/DECODE, COUNT/SUM/AVG/MIN/MAX, ROLLUP/CUBE/GROUPING SETS, ROW_NUMBER/RANK/DENSE_RANK/LAG/LEAD, SUBSTR, TO_CHAR/TO_DATE, REGEXP 계열

빨간색 함수 = 복원문제·공개 후기에서 반복 확인되거나 공식 범위상 결과 예측 문제가 자주 만들어지는 고빈도 함수입니다.

3-1. 함수 기본

[단일행 함수와 집계 함수]

정의: 단일행 함수는 행마다 하나의 결과를 만들고, 집계 함수는 여러 행을 모아 하나의 결과를 만든다.

핵심 설명: LOWER, SUBSTR, ROUND, NVL 같은 함수는 각 행에 적용됩니다. COUNT, SUM, AVG 같은 집계 함수는 여러 행을 묶어 계산합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       LOWER(email) AS email_lower,    -- 단일행 함수  
       salary  
FROM HR.EMPLOYEES  
WHERE department_id = 60;
```

```
SELECT department_id,  
       COUNT(*) AS cnt,                -- 집계 함수  
       AVG(salary) AS avg_salary  
FROM HR.EMPLOYEES  
GROUP BY department_id;
```

★ **시험 포인트:** 단일행 함수는 행 수를 줄이지 않고, GROUP BY 집계는 그룹별 결과로 행 수를 줄입니다.

3-2. 문자 함수

[UPPER]

정의: 문자를 대문자로 바꾼다.

핵심 설명: UPPER 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, UPPER(last_name) AS upper_name
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 대소문자 비교나 출력 형식 통일에 사용합니다.

[LOWER]

정의: 문자를 소문자로 바꾼다.

핵심 설명: LOWER 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, LOWER(email) AS lower_email
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 이메일처럼 대소문자를 통일할 때 사용합니다.

[INITCAP]

정의: 단어의 첫 글자를 대문자로 바꾸고 나머지를 소문자로 바꾼다.

핵심 설명: INITCAP 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, INITCAP(last_name) AS initcap_name
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 이름 출력 형식을 다듬을 때 사용합니다.

[CONCAT / ||]

정의: 문자열을 이어 붙인다.

핵심 설명: CONCAT / || 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       first_name || ' ' || last_name AS full_name,  
       CONCAT(first_name, last_name) AS concat_name  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Oracle 에서는 두 개 이상 문자열 연결 시 ||가 더 편합니다.

[SUBSTR] ● 출제 빈도 높음

정의: 문자열 일부를 잘라낸다.

핵심 설명: SUBSTR 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       phone_number,  
       SUBSTR(phone_number, 1, 3) AS phone_prefix  
FROM HR.EMPLOYEES  
WHERE phone_number IS NOT NULL  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Oracle SUBSTR 의 시작 위치는 1 부터 시작합니다.

[LENGTH]

정의: 문자열 길이를 반환한다.

핵심 설명: LENGTH 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, LENGTH(last_name) AS name_len  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Oracle LENGTH 는 문자 길이를 반환합니다.

[INSTR]

정의: 찾는 문자열이 처음 나타나는 위치를 반환한다.

핵심 설명: INSTR 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, email, INSTR(email, 'A') AS pos_a
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 없으면 0 을 반환합니다.

[LPAD]

정의: 왼쪽을 특정 문자로 채워 길이를 맞춘다.

핵심 설명: LPAD 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id,
       LPAD(employee_id, 6, '0') AS padded_emp_id
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 코드 자리수 맞춤 문제에 자주 쓰입니다.

[RPAD]

정의: 오른쪽을 특정 문자로 채워 길이를 맞춘다.

핵심 설명: RPAD 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT job_id,
       RPAD(job_id, 15, '.') AS job_id_pad
FROM HR.JOBS
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 출력 폭을 맞추는 데 사용합니다.

[TRIM]

정의: 앞뒤 공백 또는 지정 문자를 제거한다.

핵심 설명: TRIM 함수는 HR의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT TRIM(' SQLD ') AS trimmed_text
FROM DUAL;
```

★ **시험 포인트:** 앞뒤 공백 제거의 기본 함수입니다.

[LTRIM / RTRIM]

정의: 왼쪽 또는 오른쪽의 공백/문자를 제거한다.

핵심 설명: LTRIM / RTRIM 함수는 HR의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT LTRIM('000123', '0') AS left_trimmed,
       RTRIM('SQLDxxx', 'x') AS right_trimmed
FROM DUAL;
```

★ **시험 포인트:** 특정 문자를 제거할 수 있습니다.

[REPLACE]

정의: 특정 문자열을 다른 문자열로 바꾼다.

핵심 설명: REPLACE 함수는 HR의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT phone_number,
       REPLACE(phone_number, '-', '-') AS phone_dash
FROM HR.EMPLOYEES
WHERE phone_number IS NOT NULL
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 문자열 치환 결과를 묻는 문제에 주의합니다.

[TRANSLATE]

정의: 문자 단위로 여러 문자를 동시에 치환한다.

핵심 설명: TRANSLATE 함수는 HR 의 이름, 이메일, 전화번호 같은 문자 컬럼을 가공할 때 사용합니다.

HR 예시 쿼리:

```
SELECT TRANSLATE('ABC-123', 'ABC-', 'abc_') AS translated
FROM DUAL;
```

★ **시험 포인트:** REPLACE 는 문자열 단위, TRANSLATE 는 문자 단위 치환입니다.

3-3. 숫자 함수

[ROUND] 출제 빈도 높음

정의: 지정한 자리에서 반올림한다.

핵심 설명: ROUND 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, salary,
       ROUND(salary / 12, 2) AS monthly_salary_rounded
FROM HR.EMPLOYEES
WHERE salary IS NOT NULL
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** ROUND(값, 2)는 소수 둘째 자리까지 반올림합니다.

[TRUNC] 출제 빈도 높음

정의: 지정한 자리에서 버림한다.

핵심 설명: TRUNC 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, salary,
       TRUNC(salary / 12, 2) AS monthly_salary_trunc
```

```
FROM HR.EMPLOYEES
WHERE salary IS NOT NULL
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Oracle 은 숫자 버림 함수명이 TRUNC 입니다. MySQL 의 TRUNCATE 와 다릅니다.

[CEIL / CEILING]

정의: 주어진 값보다 크거나 같은 가장 작은 정수를 반환한다.

핵심 설명: CEIL / CEILING 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT CEIL(123.1) AS ceil_value,
       CEILING(123.1) AS ceiling_value
FROM DUAL;
```

★ **시험 포인트:** 올림 함수입니다.

[FLOOR]

정의: 주어진 값보다 작거나 같은 가장 큰 정수를 반환한다.

핵심 설명: FLOOR 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT FLOOR(123.9) AS floor_value
FROM DUAL;
```

★ **시험 포인트:** 내림 함수입니다.

[MOD] 출제 빈도 높음

정의: 나머지를 반환한다.

핵심 설명: MOD 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id,
       MOD(employee_id, 2) AS odd_even_flag
```

```
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 짝수/홀수 또는 주기 계산에 사용됩니다.

[ABS]

정의: 절댓값을 반환한다.

핵심 설명: ABS 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT ABS(-10000) AS abs_value  
FROM DUAL;
```

★ **시험 포인트:** 부호를 제거합니다.

[SIGN]

정의: 양수면 1, 0 이면 0, 음수면 -1 을 반환한다.

핵심 설명: SIGN 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT SIGN(100) AS pos,  
       SIGN(0) AS zero_value,  
       SIGN(-100) AS neg  
FROM DUAL;
```

★ **시험 포인트:** 값의 방향만 판단할 때 사용합니다.

[POWER]

정의: 거듭제곱을 반환한다.

핵심 설명: POWER 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT POWER(2, 3) AS power_value  
FROM DUAL;
```

★ **시험 포인트:** 2 의 3 제곱은 8 입니다.

[SQRT]

정의: 제곱근을 반환한다.

핵심 설명: SQRT 함수는 급여, 수당, 계산 금액처럼 숫자 데이터를 다룰 때 사용합니다.

HR 예시 쿼리:

```
SELECT SQRT(16) AS sqrt_value  
FROM DUAL;
```

★ **시험 포인트:** 루트 계산입니다.

3-4. 날짜 함수

[SYSDATE] 출제 빈도 높음

정의: Oracle 데이터베이스 서버의 현재 날짜와 시간을 반환한다.

핵심 설명: SYSDATE 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT SYSDATE AS now_value  
FROM DUAL;
```

★ **시험 포인트:** 현재 시각을 기준으로 날짜 계산을 할 때 사용합니다.

[CURRENT_DATE]

정의: 현재 세션 시간대 기준의 날짜를 반환한다.

핵심 설명: CURRENT_DATE 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT CURRENT_DATE AS current_date_value  
FROM DUAL;
```

★ **시험 포인트:** SYSDATE 와 시간대 기준이 다를 수 있습니다.

[ADD_MONTHS] 출제 빈도 높음

정의: 날짜에 개월 수를 더한다.

핵심 설명: ADD_MONTHS 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT employee_id, hire_date,  
       ADD_MONTHS(hire_date, 6) AS after_6_months  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 월 단위 계산은 단순 + 숫자보다 ADD_MONTHS 가 안전합니다.

[MONTHS_BETWEEN] 출제 빈도 높음

정의: 두 날짜 사이의 개월 수를 반환한다.

핵심 설명: MONTHS_BETWEEN 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT employee_id, hire_date,  
       MONTHS_BETWEEN(SYSDATE, hire_date) AS months_worked  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 근속 개월 수 계산에 사용할 수 있습니다.

[LAST_DAY] 출제 빈도 높음

정의: 해당 월의 마지막 날짜를 반환한다.

핵심 설명: LAST_DAY 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:


```
SELECT hire_date,  
       LAST_DAY(hire_date) AS hire_month_last_day  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 월말 계산 문제에 자주 쓰입니다.

[NEXT_DAY]

정의: 지정 날짜 이후의 다음 요일 날짜를 반환한다.

핵심 설명: NEXT_DAY 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT NEXT_DAY(SYSDATE, 'MONDAY') AS next_monday  
FROM DUAL;
```

★ **시험 포인트:** NLS_LANGUAGE 설정에 따라 요일 문자열 표기가 달라질 수 있습니다.

[EXTRACT] 출제 빈도 높음

정의: 날짜에서 연도, 월, 일 같은 일부 값을 추출한다.

핵심 설명: EXTRACT 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT employee_id, hire_date,  
       EXTRACT(YEAR FROM hire_date) AS hire_year,  
       EXTRACT(MONTH FROM hire_date) AS hire_month  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 연도별/월별 조건이나 집계에 사용합니다.

[TRUNC(date)] 출제 빈도 높음

정의: 날짜를 지정 단위로 버림한다.

핵심 설명: TRUNC(date) 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT hire_date,  
       TRUNC(hire_date, 'MM') AS hire_month_start,  
       TRUNC(hire_date, 'YYYY') AS hire_year_start  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 월초/연초 기준 그룹화에 유용합니다.

[ROUND(date)] ● 출제 빈도 높음

정의: 날짜를 지정 단위로 반올림한다.

핵심 설명: ROUND(date) 함수는 HIRE_DATE 같은 날짜 컬럼을 기준으로 근속 기간, 월별 집계, 기준일 계산에 사용됩니다.

HR 예시 쿼리:

```
SELECT hire_date,  
       ROUND(hire_date, 'MM') AS hire_month_rounded  
FROM HR.EMPLOYEES  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 날짜 ROUND와 숫자 ROUND는 모두 가능하지만 의미가 다릅니다.

3-5. 변환 함수

[TO_CHAR] 출제 빈도 높음

정의: 날짜나 숫자를 원하는 형식의 문자로 변환한다.

핵심 설명: TO_CHAR 함수는 문자, 숫자, 날짜 타입을 명시적으로 변환할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       TO_CHAR(hire_date, 'YYYY-MM-DD') AS hire_date_text,  
       TO_CHAR(salary, '999,999') AS salary_text  
FROM HR.EMPLOYEES
```

FETCH FIRST 5 ROWS ONLY;

★ **시험 포인트:** 출력 형식을 맞출 때 사용합니다.

오답 방지: 암시적 형변환은 문제에서 예상과 다른 결과나 오류를 만들 수 있습니다.

[TO_DATE] 출제 빈도 높음

정의: 문자를 날짜로 변환한다.

핵심 설명: TO_DATE 함수는 문자, 숫자, 날짜 타입을 명시적으로 변환할 때 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date
FROM HR.EMPLOYEES
WHERE hire_date >= TO_DATE('2007-01-01', 'YYYY-MM-DD');
```

★ **시험 포인트:** 문자 날짜 비교는 반드시 날짜 타입으로 변환하는 것이 안전합니다.

오답 방지: 암시적 형변환은 문제에서 예상과 다른 결과나 오류를 만들 수 있습니다.

[TO_NUMBER] 출제 빈도 높음

정의: 문자를 숫자로 변환한다.

핵심 설명: TO_NUMBER 함수는 문자, 숫자, 날짜 타입을 명시적으로 변환할 때 사용합니다.

HR 예시 쿼리:

```
SELECT TO_NUMBER('12000') + 500 AS amount
FROM DUAL;
```

★ **시험 포인트:** 숫자처럼 보이는 문자도 명시 변환을 알아두면 좋습니다.

오답 방지: 암시적 형변환은 문제에서 예상과 다른 결과나 오류를 만들 수 있습니다.

[CAST]

정의: 표준 SQL 방식으로 데이터 타입을 변환한다.

핵심 설명: CAST 함수는 문자, 숫자, 날짜 타입을 명시적으로 변환할 때 사용합니다.

HR 예시 쿼리:

```
SELECT CAST(employee_id AS VARCHAR2(10)) AS emp_id_text,
```

```
CAST(salary AS NUMBER(10,2)) AS salary_num
FROM HR.EMPLOYEES
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** DBMS 별 함수 차이가 있을 때 CAST 는 표준형으로 자주 등장합니다.

오답 방지: 암시적 형변환은 문제에서 예상과 다른 결과나 오류를 만들 수 있습니다.

3-6. NULL 처리 함수

[NVL] 출제 빈도 높음

정의: 첫 번째 값이 NULL 이면 두 번째 값을 반환하고, 아니면 첫 번째 값을 반환한다.

핵심 설명: NVL 함수는 NULL 때문에 계산이나 조건 결과가 달라지는 상황을 제어합니다.

HR 예시 쿼리:

```
SELECT employee_id, salary, commission_pct,
       salary + NVL(commission_pct * salary, 0) AS pay_with_commission
FROM HR.EMPLOYEES
WHERE employee_id IN (100, 145, 146);
```

★ **시험 포인트:** Oracle 빈출 함수입니다. 산술식에 NULL 이 들어가기 전에 NVL 로 대체합니다.

오답 방지: NULL 은 0 도 빈 문자열도 아니므로 일반 산술/비교 전에 처리해야 합니다.

[NVL2] 출제 빈도 높음

정의: 첫 번째 값이 NULL 이 아니면 두 번째 값, NULL 이면 세 번째 값을 반환한다.

핵심 설명: NVL2 함수는 NULL 때문에 계산이나 조건 결과가 달라지는 상황을 제어합니다.

HR 예시 쿼리:

```
SELECT employee_id, commission_pct,
       NVL2(commission_pct, 'COMMISSION', 'NO_COMMISSION') AS commission_status
FROM HR.EMPLOYEES
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** NVL2(a,b,c)는 a 가 NULL 이 아니면 b, NULL 이면 c 입니다.

오답 방지: NULL 은 0 도 빈 문자열도 아니므로 일반 산술/비교 전에 처리해야 합니다.

[COALESCE] 출제 빈도 높음

정의: 인자 중 왼쪽부터 첫 번째 NULL 이 아닌 값을 반환한다.

핵심 설명: COALESCE 함수는 NULL 때문에 계산이나 조건 결과가 달라지는 상황을 제어합니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       COALESCE(phone_number, email, 'NO_CONTACT') AS contact_info  
FROM HR.EMPLOYEES  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 여러 후보값 중 우선순위가 있는 대체값 선택에 사용합니다.

오답 방지: NULL 은 0 도 빈 문자열도 아니므로 일반 산술/비교 전에 처리해야 합니다.

[NULLIF] 출제 빈도 높음

정의: 두 값이 같으면 NULL, 다르면 첫 번째 값을 반환한다.

핵심 설명: NULLIF 함수는 NULL 때문에 계산이나 조건 결과가 달라지는 상황을 제어합니다.

HR 예시 쿼리:

```
SELECT employee_id, salary,  
       salary / NULLIF(commission_pct, 0) AS salary_divided_by_commission  
FROM HR.EMPLOYEES  
WHERE commission_pct IS NOT NULL  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** 분모가 0 이면 NULL 로 바꿔 나눗셈 오류를 피하는 예시가 자주 등장합니다.

오답 방지: NULL 은 0 도 빈 문자열도 아니므로 일반 산술/비교 전에 처리해야 합니다.

[LNNVL]

정의: 조건 결과가 FALSE 또는 UNKNOWN 이면 TRUE 를 반환한다.

핵심 설명: LNNVL 함수는 NULL 때문에 계산이나 조건 결과가 달라지는 상황을 제어합니다.

HR 예시 쿼리:

```
SELECT employee_id, commission_pct  
FROM HR.EMPLOYEES  
WHERE LNNVL(commission_pct > 0.2)
```

```
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** Oracle 함수입니다. NULL 때문에 UNKNOWN 이 되는 조건을 다룰 때 보강 개념으로 알아두면 좋습니다.

오답 방지: NULL 은 0 도 빈 문자열도 아니므로 일반 산술/비교 전에 처리해야 합니다.

3-7. 조건 함수

[CASE] 출제 빈도 높음

정의: 조건에 따라 다른 값을 반환하는 표준 SQL 조건식이다.

핵심 설명: CASE 는 IF 문처럼 위에서 아래로 조건을 평가하고 처음 만족한 THEN 값을 반환합니다. 범위 조건, 복합 조건을 표현하기 좋습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary,
       CASE
         WHEN salary >= 15000 THEN 'HIGH'
         WHEN salary >= 8000  THEN 'MIDDLE'
         ELSE 'LOW'
       END AS salary_grade
FROM HR.EMPLOYEES
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** CASE 는 범위 조건에 적합합니다.

오답 방지: 조건 순서가 중요합니다. 먼저 만족한 조건에서 멈춥니다.

[DECODE] 출제 빈도 높음

정의: Oracle 에서 표현식 값 비교에 따라 다른 값을 반환하는 조건 함수다.

핵심 설명: DECODE(expr, 비교값 1, 결과 1, 비교값 2, 결과 2, 기본값) 순서로 읽습니다. 단순 값 비교에 적합합니다.

HR 예시 쿼리:

```
SELECT employee_id, job_id,
       DECODE(job_id,
              'IT_PROG', 'IT 개발자',
```

```
'SA_REP', '영업 담당',  
'기타') AS job_group  
FROM HR.EMPLOYEES  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** DECODE 는 Oracle 빈출 함수입니다. 인자 순서를 정확히 기억해야 합니다.

3-8. 집계 함수

[COUNT(*)] 출제 빈도 높음

정의: 컬럼값의 NULL 여부와 관계없이 행 수를 센다.

핵심 설명: COUNT(*)는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT COUNT(*) AS total_employees  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 전체 행 수입니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[COUNT(컬럼)] 출제 빈도 높음

정의: 해당 컬럼이 NULL 이 아닌 행 수를 센다.

핵심 설명: COUNT(컬럼)는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT COUNT(commission_pct) AS commission_employee_count  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** COMMISSION_PCT 가 NULL 인 직원은 제외됩니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[COUNT(DISTINCT 컬럼)] 출제 빈도 높음

정의: NULL 을 제외하고 서로 다른 값의 개수를 센다.

핵심 설명: COUNT(DISTINCT 컬럼)는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT COUNT(DISTINCT department_id) AS department_count  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 중복 부서번호는 한 번만 셉니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[SUM] 출제 빈도 높음

정의: NULL 을 제외하고 합계를 구한다.

핵심 설명: SUM 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, SUM(salary) AS total_salary  
FROM HR.EMPLOYEES  
GROUP BY department_id  
ORDER BY department_id;
```

★ **시험 포인트:** 급여 합계 계산에 사용됩니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[AVG] 출제 빈도 높음

정의: NULL 을 제외하고 평균을 구한다.

핵심 설명: AVG 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, AVG(salary) AS avg_salary  
FROM HR.EMPLOYEES  
GROUP BY department_id  
ORDER BY department_id;
```

★ **시험 포인트:** NULL 인 급여가 있으면 평균 계산에서 제외됩니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[MIN] 출제 빈도 높음

정의: 최솟값을 구한다.

핵심 설명: MIN 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, MIN(salary) AS min_salary
FROM HR.EMPLOYEES
GROUP BY department_id
ORDER BY department_id;
```

★ **시험 포인트:** 날짜 컬럼에 쓰면 가장 이른 날짜를 찾습니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[MAX] 출제 빈도 높음

정의: 최댓값을 구한다.

핵심 설명: MAX 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, MAX(salary) AS max_salary
FROM HR.EMPLOYEES
GROUP BY department_id
ORDER BY department_id;
```

★ **시험 포인트:** 날짜 컬럼에 쓰면 가장 늦은 날짜를 찾습니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[STDDEV]

정의: 표준편차를 구한다.

핵심 설명: STDDEV 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, STDDEV(salary) AS salary_stddev
FROM HR.EMPLOYEES
GROUP BY department_id
ORDER BY department_id;
```

★ **시험 포인트:** 분산/표준편차는 통계 집계 함수입니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[VARIANCE]

정의: 분산을 구한다.

핵심 설명: VARIANCE 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id, VARIANCE(salary) AS salary_variance
FROM HR.EMPLOYEES
GROUP BY department_id
ORDER BY department_id;
```

★ **시험 포인트:** SQLD 에서는 COUNT/SUM/AVG/MIN/MAX 가 더 중요하지만 통계 집계도 알아두면 좋습니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

[LISTAGG]

정의: 그룹 안의 문자열 값을 하나의 문자열로 이어 붙인다.

핵심 설명: LISTAGG 는 여러 행을 모아 하나의 값을 계산하는 집계 함수입니다.

HR 예시 쿼리:

```
SELECT department_id,
       LISTAGG(last_name, ', ' ) WITHIN GROUP (ORDER BY last_name) AS employee_names
FROM HR.EMPLOYEES
GROUP BY department_id
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Oracle 문자열 집계 함수입니다. SQLD 핵심보다는 보강용으로 봅니다.

오답 방지: 집계 함수는 대부분 NULL 을 제외하지만 COUNT(*)는 행 자체를 셉니다.

4 절 WHERE 절

[WHERE 절]

정의: 조회 대상 행을 조건으로 필터링하는 절이다.

핵심 설명: WHERE 는 GROUP BY 보다 먼저 실행되어 개별 행 단위 조건을 적용합니다. TRUE 인 행만 남고 FALSE 또는 UNKNOWN 인 행은 제외됩니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE salary >= 10000;
```

★ **시험 포인트:** WHERE 는 행 조건, HAVING 은 그룹 조건입니다.

[비교 연산자]

정의: =, <>, !=, >, >=, <, <=로 값을 비교한다.

핵심 설명: 비교 연산자는 TRUE/FALSE/UNKNOWN 을 반환합니다. NULL 은 일반 비교 연산자로 비교하지 않고 IS NULL 을 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE salary <> 10000
AND salary >= 8000;
```

★ **시험 포인트:** NULL = NULL 은 TRUE 가 아닙니다.

오답 방지: NULL 비교는 IS NULL 또는 IS NOT NULL 을 사용하세요.

[BETWEEN]

정의: 시작값 이상 끝값 이하 범위에 포함되는지 검사한다.

핵심 설명: BETWEEN 은 양 끝값을 모두 포함합니다. 숫자와 날짜 범위 조건에서 자주 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE salary BETWEEN 5000 AND 10000;
```

★ **시험 포인트:** BETWEEN 은 이상/이하입니다.

오답 방지: 초과/미만이 필요하다면 >, <를 직접 사용하세요.

[IN]

정의: 여러 후보값 중 하나와 일치하는지 검사한다.

핵심 설명: IN 은 여러 OR 조건을 간단히 쓰는 형태입니다. 서브쿼리 결과 집합과 비교할 때도 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id
FROM HR.EMPLOYEES
WHERE department_id IN (50, 60, 80);
```

★ **시험 포인트:** IN 은 목록 중 하나와 같으면 TRUE 입니다.

오답 방지: NOT IN 은 목록에 NULL 이 있으면 결과가 예상과 달라질 수 있습니다.

[LIKE]

정의: 문자 패턴과 일치하는지 검사한다.

핵심 설명: LIKE 에서 %는 0 개 이상의 임의 문자, _는 정확히 한 글자를 의미합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name
FROM HR.EMPLOYEES
WHERE last_name LIKE 'S%';
```

★ **시험 포인트:** %와 _ 의미를 구분해야 합니다.

오답 방지: LIKE '%S%'처럼 앞에 %가 있으면 일반적으로 인덱스 활용이 어려워질 수 있습니다.

[ESCAPE]

정의: LIKE 에서 %, _를 일반 문자로 취급하게 한다.

핵심 설명: 검색하려는 문자 자체가 % 또는 _이면 ESCAPE 문자를 지정해 와일드카드가 아님을 표시합니다.

HR 예시 쿼리:

```
SELECT employee_id, email
FROM HR.EMPLOYEES
WHERE email LIKE '%W_%' ESCAPE 'W';
```

★ **시험 포인트:** 와일드카드 문자를 문자 그대로 찾는 방법입니다.

[논리 연산자]

정의: AND, OR, NOT 으로 조건을 결합한다.

핵심 설명: 일반적인 우선순위는 NOT, AND, OR 입니다. 괄호가 없으면 AND 가 OR 보다 먼저 처리됩니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary, department_id
FROM HR.EMPLOYEES
WHERE department_id = 60
      OR department_id = 80
      AND salary >= 10000;
```

★ **시험 포인트:** AND 가 OR 보다 먼저 처리됩니다.

오답 방지: 의도와 다르다면 괄호를 사용해야 합니다.

[NULL 조건]

정의: NULL 여부는 IS NULL 또는 IS NOT NULL 로 판단한다.

핵심 설명: NULL 은 값이 없거나 모르는 상태입니다. 일반 비교 결과가 UNKNOWN 이 될 수 있고 WHERE 는 UNKNOWN 을 남기지 않습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, commission_pct
FROM HR.EMPLOYEES
WHERE commission_pct IS NULL;
```

★ **시험 포인트:** NULL 관련 문제는 WHERE, OUTER JOIN, NOT IN 에서 자주 나옵니다.

오답 방지: commission_pct = NULL 은 올바른 NULL 판단 방식이 아닙니다.

5 절 GROUP BY, HAVING 절

[GROUP BY]

정의: 행을 특정 컬럼 기준으로 묶어 그룹별 결과를 만드는 절이다.

핵심 설명: GROUP BY 는 같은 값을 가진 행을 하나의 그룹으로 묶습니다. 그룹별 COUNT, SUM, AVG 를 구할 때 사용합니다.

HR 예시 쿼리:

```
SELECT department_id,  
       COUNT(*) AS emp_count,  
       ROUND(AVG(salary), 2) AS avg_salary  
FROM HR.EMPLOYEES  
GROUP BY department_id  
ORDER BY department_id;
```

★ **시험 포인트:** SELECT 에 집계 함수가 아닌 컬럼을 쓰려면 일반적으로 GROUP BY 에 포함되어야 합니다.

오답 방지: department_name 을 함께 보고 싶다면 DEPARTMENTS 와 JOIN 하거나 GROUP BY 에 포함해야 합니다.

[HAVING]

정의: 그룹화된 결과에 조건을 적용하는 절이다.

핵심 설명: WHERE 는 그룹화 전 행을 필터링하고, HAVING 은 GROUP BY 이후 그룹을 필터링합니다. 집계 함수 조건은 HAVING 에 씁니다.

HR 예시 쿼리:

```
SELECT department_id,  
       COUNT(*) AS emp_count,  
       AVG(salary) AS avg_salary  
FROM HR.EMPLOYEES  
GROUP BY department_id  
HAVING COUNT(*) >= 5  
ORDER BY emp_count DESC;
```

★ **시험 포인트:** 집계 조건은 WHERE 가 아니라 HAVING 입니다.

오답 방지: WHERE COUNT(*) >= 5 는 논리적으로 맞지 않습니다.

[WHERE 와 HAVING 차이]

정의: WHERE 는 행 조건, HAVING 은 그룹 조건이다.

핵심 설명: 개별 사원 급여 조건은 WHERE 에 쓰고, 부서 평균 급여 조건은 HAVING 에 씁니다. 가능한 행 조건은 WHERE 에서 먼저 거르는 것이 일반적으로 효율적입니다.

HR 예시 쿼리:

```
SELECT department_id,  
       AVG(salary) AS avg_salary  
FROM HR.EMPLOYEES  
WHERE salary >= 3000  
GROUP BY department_id  
HAVING AVG(salary) >= 7000;
```

★ **시험 포인트:** 처리 순서 때문에 WHERE 와 HAVING 은 결과가 달라질 수 있습니다.

[GROUP BY 와 NULL]

정의: GROUP BY 대상 컬럼의 NULL 은 하나의 그룹으로 묶일 수 있다.

핵심 설명: NULL 은 비교에서는 UNKNOWN 을 만들 수 있지만, GROUP BY 에서는 NULL 값들이 하나의 그룹처럼 묶입니다.

HR 예시 쿼리:

```
SELECT department_id,  
       COUNT(*) AS emp_count  
FROM HR.EMPLOYEES  
GROUP BY department_id  
ORDER BY department_id NULLS FIRST;
```

★ **시험 포인트:** NULL 그룹이 생길 수 있음을 기억합니다.

오답 방지: NULL 은 비교와 그룹화에서 체감이 다릅니다.

6 절 ORDER BY 절

[ORDER BY]

정의: 최종 조회 결과의 표시 순서를 정렬하는 절이다.

핵심 설명: ORDER BY 는 결과 표시 순서를 정할 뿐, 테이블의 실제 저장 순서를 바꾸지는 않습니다. 기본 정렬 방향은 ASC 입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
ORDER BY salary DESC, employee_id ASC
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 동점이 있으면 추가 정렬 기준이 없을 때 순서가 보장되지 않을 수 있습니다.

[ASC 와 DESC]

정의: ASC 는 오름차순, DESC 는 내림차순이다.

핵심 설명: 날짜 DESC 는 최신 날짜가 먼저 나오고, 숫자 DESC 는 큰 값이 먼저 나옵니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date
FROM HR.EMPLOYEES
ORDER BY hire_date DESC
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** ASC 는 생략 가능하지만 DESC 는 명시해야 합니다.

[ORDER BY 에서 별칭 사용]

정의: SELECT 에서 만든 별칭으로 정렬할 수 있다.

핵심 설명: ORDER BY 는 SELECT 목록 계산 이후 처리되므로 SELECT 별칭을 사용할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id,
```



```
last_name,  
salary * 12 AS annual_salary  
FROM HR.EMPLOYEES  
ORDER BY annual_salary DESC  
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** WHERE 에서는 SELECT 별칭 사용이 어렵지만 ORDER BY 에서는 가능합니다.

[NULL 정렬]

정의: NULLS FIRST 또는 NULLS LAST 로 NULL 위치를 지정할 수 있다.

핵심 설명: Oracle 에서는 정렬 방향에 따라 NULL 기본 위치가 달라질 수 있으므로, 시험에서는 NULLS FIRST/LAST 가 명시되면 그대로 적용해야 합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, commission_pct  
FROM HR.EMPLOYEES  
ORDER BY commission_pct DESC NULLS LAST;
```

★ **시험 포인트:** NULL 정렬 기준이 명시되면 반드시 확인합니다.

오답 방지: DBMS 마다 NULL 기본 정렬 위치가 다를 수 있습니다.

7 절 조인과 표준 조인

[JOIN]

정의: 두 개 이상의 테이블을 관련 컬럼 기준으로 연결해 조회하는 것이다.

핵심 설명: JOIN 은 1 과목의 관계가 SQL 에서 표현되는 방식입니다. 보통 부모 테이블의 PK 와 자식 테이블의 FK 를 조건으로 연결합니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** JOIN 결과 행 수는 관계차수와 매칭 여부에 따라 달라집니다.

[INNER JOIN]

정의: 양쪽 테이블에 매칭되는 행만 반환하는 조인이다.

핵심 설명: 조인 조건을 만족하는 행만 결과에 남기므로 한쪽에만 있는 데이터는 제외됩니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
INNER JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id;
```

★ **시험 포인트:** 기본 조인 결과 행 수는 매칭되는 조합 수를 봐야 합니다.

오답 방지: 부서가 없는 사원은 INNER JOIN 결과에서 빠질 수 있습니다.

[LEFT OUTER JOIN]

정의: 왼쪽 테이블의 행을 보존하고 매칭이 없으면 오른쪽 컬럼에 NULL 을 붙인다.

핵심 설명: 사원은 모두 보고, 부서 정보가 있으면 붙이고 싶을 때 사용합니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name
```

```
FROM HR.EMPLOYEES e
LEFT OUTER JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id;
```

★ **시험 포인트:** OUTER JOIN 결과에서 매칭되지 않은 상대 테이블 컬럼은 NULL 입니다.

[RIGHT OUTER JOIN]

정의: 오른쪽 테이블의 행을 보존하는 외부 조인이다.

핵심 설명: 부서를 모두 보고, 해당 부서에 속한 사원이 있으면 붙이고 싶을 때 사용할 수 있습니다.

HR 예시 쿼리:

```
SELECT d.department_id, d.department_name, e.employee_id, e.last_name
FROM HR.EMPLOYEES e
RIGHT OUTER JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id;
```

★ **시험 포인트:** 어느 쪽 테이블이 보존되는지 확인해야 합니다.

오답 방지: LEFT JOIN 으로 바꿔도 같은 의미를 표현할 수 있습니다.

[FULL OUTER JOIN]

정의: 양쪽 테이블의 모든 행을 보존하는 외부 조인이다.

핵심 설명: 매칭되는 행은 합치고, 어느 한쪽에만 있는 행도 결과에 포함합니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
FULL OUTER JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id;
```

★ **시험 포인트:** FULL OUTER JOIN 은 양쪽 미매칭 행을 모두 보존합니다.

[ON 조건과 WHERE 조건]

정의: JOIN 조건과 결과 필터 조건의 위치 차이이다.

핵심 설명: OUTER JOIN 에서 오른쪽 테이블 조건을 WHERE 에 두면 보존되어야 할 NULL 행이 제거될 수 있습니다.

HR 예시 쿼리:

```
-- 부서가 없는 사원도 보존하면서, 60 번 부서 정보만 붙이는 예
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
LEFT JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id
 AND d.department_id = 60;

-- WHERE 에 두면 60 번 부서가 아닌 사원과 부서 없는 사원이 제거됨
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
LEFT JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id
WHERE d.department_id = 60;
```

★ **시험 포인트:** OUTER JOIN 에서 조건 위치는 매우 중요합니다.

오답 방지: LEFT JOIN 후 WHERE 오른쪽테이블.컬럼 = 값 조건은 INNER JOIN 처럼 동작할 수 있습니다.

[CROSS JOIN]

정의: 두 테이블의 모든 행 조합을 만드는 조인이다.

핵심 설명: 조인 조건 없이 가능한 모든 조합을 만듭니다. 이를 카테시안 곱이라고 합니다.

HR 예시 쿼리:

```
SELECT e.employee_id, j.job_id
FROM HR.EMPLOYEES e
CROSS JOIN HR.JOBS j
WHERE e.employee_id IN (100, 101);
```

★ **시험 포인트:** 조건을 빠뜨린 조인은 의도치 않은 대량 결과를 만들 수 있습니다.

[NATURAL JOIN]

정의: 같은 이름의 컬럼을 자동으로 조인 조건에 사용하는 조인이다.

핵심 설명: 편리하지만 동일한 이름의 모든 컬럼을 자동으로 조건에 사용하므로 의도하지 않은 조건이 추가될 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_name
FROM HR.EMPLOYEES
NATURAL JOIN HR.DEPARTMENTS
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** NATURAL JOIN 은 같은 이름 컬럼 전체를 조인 조건으로 사용할 수 있습니다.

오답 방지: 같은 이름의 컬럼이 여러 개 있으면 예상과 다르게 결과가 줄어 들 수 있습니다.

[USING]

정의: 같은 이름의 조인 컬럼을 명시하는 표준 조인 문법이다.

핵심 설명: 두 테이블의 조인 컬럼명이 같을 때 USING 으로 간단히 작성할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_name
FROM HR.EMPLOYEES
JOIN HR.DEPARTMENTS USING (department_id)
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** USING 컬럼은 테이블 별칭을 붙여 SELECT 하지 않는 것이 일반적입니다.

[SELF JOIN]

정의: 같은 테이블을 서로 다른 별칭으로 두 번 사용해 조인하는 것이다.

핵심 설명: 사원과 관리자처럼 같은 테이블 안의 행끼리 관계가 있을 때 사용합니다.

HR 예시 쿼리:

```
SELECT e.employee_id,
       e.last_name AS employee_name,
       m.last_name AS manager_name
FROM HR.EMPLOYEES e
LEFT JOIN HR.EMPLOYEES m
      ON e.manager_id = m.employee_id
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 같은 테이블을 조인하므로 별칭으로 역할을 구분해야 합니다.

[NON-EQUI JOIN]

정의: =이 아닌 범위 조건 등으로 연결하는 조인이다.

핵심 설명: 급여가 직무의 최소/최대 급여 범위 안에 있는지처럼 BETWEEN, >=, <= 조건으로 조인할 수 있습니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, e.salary,  
       j.job_title, j.min_salary, j.max_salary  
FROM HR.EMPLOYEES e  
JOIN HR.JOBS j  
      ON e.salary BETWEEN j.min_salary AND j.max_salary  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 조인 조건이 항상 =인 것은 아닙니다.

[Oracle 외부 조인 (+)]

정의: Oracle 전통 문법에서 외부 조인을 표시하는 기호다.

핵심 설명: 오래된 Oracle 문제나 복원 문제에서 (+) 표기가 나올 수 있습니다. (+)는 선택되는 쪽, 즉 NULL로 채워질 수 있는 테이블 컬럼에 붙습니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, d.department_name  
FROM HR.EMPLOYEES e, HR.DEPARTMENTS d  
WHERE e.department_id = d.department_id(+);
```

★ **시험 포인트:** 위 예시는 EMPLOYEES를 보존하는 LEFT OUTER JOIN과 비슷합니다.

오답 방지: (+) 위치를 반대로 붙이면 보존되는 테이블 방향이 달라집니다.

8 절 서브쿼리

[서브쿼리]

정의: 하나의 SQL 문 안에 포함된 또 다른 SELECT 문이다.

핵심 설명: 바깥 SQL 이 필요한 값을 내부 SELECT 로 구해 사용하는 방식입니다. 서브쿼리 반환 행 수에 따라 사용할 연산자가 달라집니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE salary > (SELECT AVG(salary) FROM HR.EMPLOYEES);
```

★ **시험 포인트:** 단일행 비교 연산자에는 서브쿼리가 한 값만 반환해야 합니다.

[스칼라 서브쿼리]

정의: 한 행, 한 컬럼의 단일 값을 반환하는 서브쿼리다.

핵심 설명: 단일 값처럼 SELECT 목록이나 WHERE 절에 사용할 수 있습니다. 여러 행을 반환하면 오류가 납니다.

HR 예시 쿼리:

```
SELECT e.employee_id,
       e.last_name,
       (SELECT d.department_name
        FROM HR.DEPARTMENTS d
        WHERE d.department_id = e.department_id) AS department_name
FROM HR.EMPLOYEES e
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 스칼라 서브쿼리는 반드시 한 값만 반환해야 합니다.

[인라인 뷰]

정의: FROM 절에 사용되는 서브쿼리다.

핵심 설명: 서브쿼리 결과를 임시 테이블처럼 사용합니다. 먼저 집계한 결과를 다시 조인하거나 필터링할 때 유용합니다.

HR 예시 쿼리:

```
SELECT a.department_id, d.department_name, a.avg_salary
```

```
FROM (  
    SELECT department_id, AVG(salary) AS avg_salary  
    FROM HR.EMPLOYEES  
    GROUP BY department_id  
) a  
LEFT JOIN HR.DEPARTMENTS d  
    ON a.department_id = d.department_id  
ORDER BY a.avg_salary DESC;
```

★ **시험 포인트:** FROM 절 서브쿼리에는 별칭을 붙이는 습관이 좋습니다.

[중첩 서브쿼리]

정의: WHERE 또는 HAVING 절 안에 사용되는 서브쿼리다.

핵심 설명: 조건에 필요한 값이나 목록을 내부 SELECT 로 구합니다. IN, EXISTS, ANY, ALL 과 자주 함께 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id  
FROM HR.EMPLOYEES  
WHERE department_id IN (  
    SELECT department_id  
    FROM HR.DEPARTMENTS  
    WHERE location_id = 1700  
);
```

★ **시험 포인트:** 서브쿼리 결과가 여러 행이면 = 대신 IN 등을 고려합니다.

[상관 서브쿼리]

정의: 서브쿼리가 바깥 쿼리의 컬럼을 참조하는 서브쿼리다.

핵심 설명: 내부 쿼리가 외부 쿼리의 현재 행 값을 참조합니다. 행마다 내부 쿼리가 평가되는 구조로 이해하면 됩니다.

HR 예시 쿼리:

```
SELECT e.employee_id, e.last_name, e.salary, e.department_id  
FROM HR.EMPLOYEES e
```



```
WHERE e.salary > (  
    SELECT AVG(e2.salary)  
    FROM HR.EMPLOYEES e2  
    WHERE e2.department_id = e.department_id  
);
```

★ **시험 포인트:** 상관 서브쿼리는 외부 쿼리와 내부 쿼리의 연결 조건을 확인해야 합니다.

[IN]

정의: 서브쿼리 결과 목록 중 하나와 일치하는지 확인한다.

핵심 설명: 서브쿼리가 여러 값을 반환할 때 자주 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, job_id  
FROM HR.EMPLOYEES  
WHERE job_id IN (  
    SELECT job_id  
    FROM HR.JOBS  
    WHERE min_salary >= 6000  
);
```

★ **시험 포인트:** 목록 중 하나와 같으면 TRUE 입니다.

[ANY]

정의: 서브쿼리 결과 중 하나라도 비교 조건을 만족하면 TRUE 다.

핵심 설명: > ANY 는 최솟값보다 크면 참으로 해석할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary  
FROM HR.EMPLOYEES  
WHERE salary > ANY (  
    SELECT salary  
    FROM HR.EMPLOYEES  
    WHERE department_id = 60  
);
```

★ **시험 포인트:** ANY 는 하나라도 만족입니다.

오답 방지: > ANY 는 보통 MIN 보다 크면 참, < ANY 는 보통 MAX 보다 작으면 참으로 해석합니다.

[ALL]

정의: 서브쿼리 결과의 모든 값과 비교해 조건을 만족해야 TRUE 다.

핵심 설명: > ALL 은 최댓값보다 커야 참으로 해석할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE salary > ALL (
    SELECT salary
    FROM HR.EMPLOYEES
    WHERE department_id = 60
);
```

★ **시험 포인트:** ALL 은 모두 만족입니다.

오답 방지: > ALL 은 보통 MAX 보다 커야 참, < ALL 은 보통 MIN 보다 작아야 참입니다.

[EXISTS]

정의: 서브쿼리 결과 행이 존재하는지 확인한다.

핵심 설명: EXISTS 는 서브쿼리 결과의 값 자체보다 행 존재 여부를 봅니다. SELECT 1 을 쓰더라도 행이 있으면 TRUE 입니다.

HR 예시 쿼리:

```
SELECT d.department_id, d.department_name
FROM HR.DEPARTMENTS d
WHERE EXISTS (
    SELECT 1
    FROM HR.EMPLOYEES e
    WHERE e.department_id = d.department_id
);
```

★ **시험 포인트:** EXISTS 는 상관 서브쿼리와 함께 자주 등장합니다.

[NOT EXISTS]

정의: 서브쿼리 결과 행이 존재하지 않는지 확인한다.

핵심 설명: NOT IN + NULL 함정을 피하기 위해 NOT EXISTS 를 사용하는 경우가 많습니다.

HR 예시 쿼리:

```
SELECT d.department_id, d.department_name
FROM HR.DEPARTMENTS d
WHERE NOT EXISTS (
    SELECT 1
    FROM HR.EMPLOYEES e
    WHERE e.department_id = d.department_id
);
```

★ **시험 포인트:** 없음을 찾을 때 안전한 방식입니다.

[NOT IN 과 NULL 함정]

정의: NOT IN 비교 대상에 NULL 이 있으면 결과가 예상과 달라질 수 있다.

핵심 설명: NOT IN 은 목록의 모든 값과 같지 않아야 하는데 목록에 NULL 이 있으면 비교 결과가 UNKNOWN 이 되어 전체 결과가 비어 보일 수 있습니다.

HR 예시 쿼리:

```
-- MANAGER_ID 에는 NULL 이 있을 수 있어 결과가 비어 보일 수 있음
SELECT employee_id, last_name
FROM HR.EMPLOYEES
WHERE employee_id NOT IN (
    SELECT manager_id
    FROM HR.EMPLOYEES
);

-- 안전한 방식
SELECT e.employee_id, e.last_name
FROM HR.EMPLOYEES e
WHERE NOT EXISTS (
```

```
SELECT 1  
FROM HR.EMPLOYEES m  
WHERE m.manager_id = e.employee_id  
);
```

★ **시험 포인트:** NOT IN + NULL 은 SQLD 대표 함정입니다.

오답 방지: 서브쿼리에서 WHERE manager_id IS NOT NULL 을 추가하거나 NOT EXISTS 를 사용하세요.

9 절 집합 연산자

[집합 연산자]

정의: 두 SELECT 결과 집합을 위아래로 결합하거나 비교하는 연산자다.

핵심 설명: JOIN 이 테이블을 좌우로 연결한다면, 집합 연산자는 SELECT 결과를 위아래로 합치거나 공통/차이를 구합니다. 컬럼 개수와 데이터 타입 호환이 필요합니다.

HR 예시 쿼리:

```
SELECT employee_id FROM HR.EMPLOYEES
WHERE department_id = 60
UNION
SELECT employee_id FROM HR.EMPLOYEES
WHERE job_id = 'IT_PROG';
```

★ **시험 포인트:** 집합 연산자는 JOIN 과 다릅니다. 결과 집합끼리 연산합니다.

[UNION]

정의: 두 결과를 합치고 중복을 제거한다.

핵심 설명: UNION 은 DISTINCT 성격이 있어 같은 행은 한 번만 남깁니다.

HR 예시 쿼리:

```
SELECT job_id FROM HR.EMPLOYEES WHERE department_id = 60
UNION
SELECT job_id FROM HR.EMPLOYEES WHERE department_id = 80;
```

★ **시험 포인트:** UNION 은 중복 제거입니다.

[UNION ALL]

정의: 두 결과를 합치고 중복도 그대로 보존한다.

핵심 설명: 중복 제거가 필요 없으면 UNION ALL 이 일반적으로 더 단순합니다.

HR 예시 쿼리:

```
SELECT job_id FROM HR.EMPLOYEES WHERE department_id = 60
UNION ALL
SELECT job_id FROM HR.EMPLOYEES WHERE department_id = 80;
```

★ **시험 포인트:** UNION ALL 은 중복 보존입니다.

[INTERSECT]

정의: 두 결과 집합에 공통으로 존재하는 행을 반환한다.

핵심 설명: 양쪽 SELECT 결과에 모두 있는 행만 반환합니다.

HR 예시 쿼리:

```
SELECT job_id FROM HR.EMPLOYEES WHERE department_id = 60  
  
INTERSECT  
  
SELECT job_id FROM HR.EMPLOYEES WHERE salary >= 5000;
```

★ **시험 포인트:** INTERSECT 는 교집합입니다.

[MINUS]

정의: 앞 결과에서 뒤 결과를 뺀 차집합을 반환한다.

핵심 설명: Oracle 은 EXCEPT 대신 MINUS 를 사용합니다. 방향이 매우 중요합니다.

HR 예시 쿼리:

```
SELECT employee_id FROM HR.EMPLOYEES  
  
MINUS  
  
SELECT employee_id FROM HR.JOB_HISTORY;
```

★ **시험 포인트:** A MINUS B 와 B MINUS A 는 결과가 다릅니다.

오답 방지: Oracle 에서는 EXCEPT 보다 MINUS 가 출제될 가능성이 높습니다.

[집합 연산자의 ORDER BY]

정의: 전체 집합 연산 결과 맨 뒤에 ORDER BY 를 적용한다.

핵심 설명: 각 SELECT 마다 ORDER BY 를 쓰는 것이 아니라 전체 결과의 마지막에 정렬을 둡니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name FROM HR.EMPLOYEES WHERE department_id = 60  
  
UNION ALL  
  
SELECT employee_id, last_name FROM HR.EMPLOYEES WHERE department_id = 80
```

```
ORDER BY employee_id;
```

★ **시험 포인트:** ORDER BY 는 전체 집합 결과 마지막에 작성합니다.

[집합 연산 조건]

정의: SELECT 목록의 컬럼 개수와 대응 데이터 타입이 호환되어야 한다.

핵심 설명: 첫 번째 SELECT 의 첫 컬럼은 두 번째 SELECT 의 첫 컬럼과 호환되어야 합니다. 컬럼명은 보통 첫 번째 SELECT 를 따릅니다.

HR 예시 쿼리:

```
-- 올바른 예: 컬럼 개수 2 개, 타입 호환
SELECT employee_id, last_name FROM HR.EMPLOYEES
UNION
SELECT employee_id, last_name FROM HR.EMPLOYEES;

-- 잘못된 예: 컬럼 개수가 다름
-- SELECT employee_id, last_name FROM HR.EMPLOYEES
-- UNION
-- SELECT department_id FROM HR.DEPARTMENTS;
```

★ **시험 포인트:** 컬럼 개수와 데이터 타입 호환은 필수입니다.

10 절 그룹 함수

[그룹 함수 개요]

정의: ROLLUP, CUBE, GROUPING SETS 처럼 소계와 총계를 만드는 집계 확장 기능이다.

핵심 설명: 단순 GROUP BY 가 하나의 그룹 조합만 만든다면, 그룹 함수는 여러 수준의 소계와 전체 합계를 함께 만듭니다.

HR 예시 쿼리:

```
SELECT department_id, job_id, SUM(salary) AS total_salary
FROM HR.EMPLOYEES
GROUP BY department_id, job_id
ORDER BY department_id, job_id;
```

★ **시험 포인트:** ROLLUP, CUBE, GROUPING SETS 의 소계 생성 방식 차이를 구분해야 합니다.

[ROLLUP] 출제 빈도 높음

정의: 지정한 그룹 컬럼의 계층적 소계와 총계를 만든다.

핵심 설명: ROLLUP 은 컬럼 순서에 따라 오른쪽에서 왼쪽으로 소계를 만듭니다. ROLLUP(department_id, job_id)는 부서+직무별, 부서별, 전체 합계를 만듭니다.

HR 예시 쿼리:

```
SELECT department_id, job_id, SUM(salary) AS total_salary
FROM HR.EMPLOYEES
GROUP BY ROLLUP(department_id, job_id)
ORDER BY department_id, job_id;
```

★ **시험 포인트:** ROLLUP 은 컬럼 순서가 중요합니다.

오답 방지: ROLLUP(a,b)와 ROLLUP(b,a)는 소계 구조가 다릅니다.

[CUBE] 출제 빈도 높음

정의: 지정한 그룹 컬럼의 가능한 모든 조합별 집계를 만든다.

핵심 설명: CUBE(department_id, job_id)는 부서+직무별, 부서별, 직무별, 전체 합계를 모두 만듭니다. ROLLUP 보다 소계가 더 많습니다.

HR 예시 쿼리:


```
SELECT department_id, job_id, SUM(salary) AS total_salary
FROM HR.EMPLOYEES
GROUP BY CUBE(department_id, job_id)
ORDER BY department_id, job_id;
```

★ **시험 포인트:** CUBE 는 가능한 모든 조합의 소계를 만듭니다.

[GROUPING SETS] 출제 빈도 높음

정의: 필요한 그룹 조합만 직접 지정해 집계한다.

핵심 설명: 불필요한 소계를 만들지 않고 원하는 집계 조합만 고를 때 사용합니다.

HR 예시 쿼리:

```
SELECT department_id, job_id, SUM(salary) AS total_salary
FROM HR.EMPLOYEES
GROUP BY GROUPING SETS (
    (department_id),
    (job_id),
    ()
)
ORDER BY department_id, job_id;
```

★ **시험 포인트:** GROUPING SETS 는 필요한 그룹 조합만 지정합니다.

[GROUPING] 출제 빈도 높음

정의: 소계 행에서 어떤 컬럼이 집계 때문에 NULL 이 되었는지 구분한다.

핵심 설명: ROLLUP/CUBE 결과의 NULL 은 실제 데이터 NULL 일 수도 있고 소계를 의미하는 NULL 일 수도 있습니다. GROUPING 은 소계 때문에 NULL 이 된 경우 1 을 반환합니다.

HR 예시 쿼리:

```
SELECT department_id,
    job_id,
    GROUPING(department_id) AS grp_dept,
    GROUPING(job_id) AS grp_job,
    SUM(salary) AS total_salary
FROM HR.EMPLOYEES
```

```
GROUP BY ROLLUP(department_id, job_id)
```

```
ORDER BY department_id, job_id;
```

★ **시험 포인트:** 소계 NULL 과 실제 NULL 을 구분하는 목적입니다.

[GROUPING_ID] 출제 빈도 높음

정의: 여러 GROUPING 결과를 하나의 숫자로 표현한다.

핵심 설명: GROUPING_ID 는 ROLLUP/CUBE 결과 행이 어떤 수준의 소계인지 구분할 때 사용합니다.

HR 예시 쿼리:

```
SELECT department_id,  
       job_id,  
       GROUPING_ID(department_id, job_id) AS grouping_id,  
       SUM(salary) AS total_salary  
FROM HR.EMPLOYEES  
GROUP BY ROLLUP(department_id, job_id)  
ORDER BY department_id, job_id;
```

★ **시험 포인트:** GROUPING_ID 는 소계 레벨 판별용 보강 함수로 이해하면 됩니다.

11 절 윈도우 함수

[윈도우 함수]

정의: 행을 유지한 상태에서 관련 행 집합을 기준으로 순위나 집계를 계산하는 함수다.

핵심 설명: GROUP BY 는 여러 행을 하나로 줄이지만, 윈도우 함수는 원래 행을 유지하면서 순위, 누적합, 이전값 같은 계산 결과를 붙입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg_salary  
FROM HR.EMPLOYEES  
ORDER BY department_id, salary DESC;
```

★ **시험 포인트:** 윈도우 함수는 행을 줄이지 않습니다.

오답 방지: GROUP BY 와 가장 큰 차이는 원래 행 수가 유지된다는 점입니다.

[PARTITION BY]

정의: 윈도우 함수를 계산할 그룹을 나누는 기준이다.

핵심 설명: 같은 부서 안에서만 순위를 매기거나 부서별 평균을 붙일 때 사용합니다. PARTITION BY 가 없으면 전체 결과가 하나의 그룹처럼 계산됩니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS dept_salary_rank  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** PARTITION BY 는 계산 범위를 나눕니다.

[ORDER BY in OVER]

정의: 윈도우 함수 계산 순서를 정하는 기준이다.

핵심 설명: 순위, LAG/LEAD, 누적합은 정렬 기준이 매우 중요합니다. ORDER BY 가 없으면 이전/다음 행의 의미가 불명확할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date,
```

```
ROW_NUMBER() OVER (ORDER BY hire_date) AS hire_order
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 윈도우 함수의 ORDER BY 는 전체 결과 정렬 ORDER BY 와 별개입니다.

[ROW_NUMBER] 출제 빈도 높음

정의: 동점이어도 서로 다른 고유 번호를 부여한다.

핵심 설명: ROW_NUMBER 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM HR.EMPLOYEES
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 동점도 1,2,3 처럼 고유 번호가 부여됩니다.

[RANK] 출제 빈도 높음

정의: 동점이면 같은 순위이고 다음 순위를 건너뛴다.

핵심 설명: RANK 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM HR.EMPLOYEES
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 100,100,90 이면 1,1,3 입니다.

[DENSE_RANK] 출제 빈도 높음

정의: 동점이면 같은 순위이고 다음 순위를 건너뛰지 않는다.

핵심 설명: DENSE_RANK 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_dense_rank  
FROM HR.EMPLOYEES  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 100,100,90 이면 1,1,2 입니다.

[NTILE]

정의: 정렬된 행을 지정한 개수의 그룹으로 나눈다.

핵심 설명: NTILE 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary,  
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 4 분위, 10 분위처럼 구간 번호를 부여합니다.

[LAG] 출제 빈도 높음

정의: 현재 행보다 이전 행의 값을 가져온다.

핵심 설명: LAG 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date,  
       LAG(hire_date) OVER (ORDER BY hire_date) AS prev_hire_date  
FROM HR.EMPLOYEES  
ORDER BY hire_date;
```

★ **시험 포인트:** 이전값 비교에 사용합니다.

[LEAD] 출제 빈도 높음

정의: 현재 행보다 다음 행의 값을 가져온다.

핵심 설명: LEAD 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date,  
       LEAD(hire_date) OVER (ORDER BY hire_date) AS next_hire_date  
FROM HR.EMPLOYEES  
ORDER BY hire_date;
```

★ **시험 포인트:** 다음값 비교에 사용합니다.

[FIRST_VALUE]

정의: 윈도우 범위의 첫 번째 값을 가져온다.

핵심 설명: FIRST_VALUE 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       FIRST_VALUE(last_name) OVER (  
           PARTITION BY department_id  
           ORDER BY salary DESC  
       ) AS highest_paid_in_dept  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 부서별 최고 급여자의 이름을 각 행에 붙일 수 있습니다.

[LAST_VALUE] 출제 빈도 높음

정의: 윈도우 범위의 마지막 값을 가져온다.

핵심 설명: LAST_VALUE 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       LAST_VALUE(last_name) OVER (  
           PARTITION BY department_id  
           ORDER BY salary DESC  
           ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
       ) AS lowest_paid_in_dept  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** LAST_VALUE 는 프레임을 지정하지 않으면 현재 행까지가 기본 범위가 되어 헛갈릴 수 있습니다.

[SUM OVER]

정의: 행을 유지하면서 집계 합계를 붙인다.

핵심 설명: SUM OVER 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       SUM(salary) OVER (PARTITION BY department_id) AS dept_total_salary  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** GROUP BY 와 달리 사원 행은 그대로 유지됩니다.

[AVG OVER]

정의: 행을 유지하면서 평균을 붙인다.

핵심 설명: AVG OVER 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary,  
       AVG(salary) OVER (PARTITION BY department_id) AS dept_avg_salary  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 개별 급여와 부서 평균을 같은 행에서 비교할 수 있습니다.

[COUNT OVER]

정의: 행을 유지하면서 건수를 붙인다.

핵심 설명: COUNT OVER 는 윈도우 함수의 대표 예시입니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id,  
       COUNT(*) OVER (PARTITION BY department_id) AS dept_emp_count  
FROM HR.EMPLOYEES;
```

★ **시험 포인트:** 부서별 인원수를 각 사원 행에 붙입니다.

[ROWS 와 RANGE] 출제 빈도 높음

정의: 윈도우 함수가 계산할 행 범위를 정하는 기준이다.

핵심 설명: ROWS 는 물리적인 행 개수 기준이고, RANGE 는 ORDER BY 값의 범위 기준입니다. 누적합에서는 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW 가 자주 등장합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, hire_date, salary,  
       SUM(salary) OVER (  
         ORDER BY hire_date  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_salary_sum  
FROM HR.EMPLOYEES  
ORDER BY hire_date;
```

★ **시험 포인트:** 프레임 범위가 달라지면 누적합 결과가 달라집니다.

오답 방지: RANGE 는 동일 ORDER BY 값의 행을 함께 처리할 수 있어 ROWS 와 결과가 다를 수 있습니다.

12 절 Top N 쿼리

[Top N 쿼리]

정의: 정렬된 결과에서 상위 N 건만 조회하는 쿼리다.

핵심 설명: Top N 은 먼저 원하는 기준으로 정렬한 뒤 일부 행만 가져오는 방식입니다. Oracle 12c 이후에는 FETCH FIRST 를 사용할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
ORDER BY salary DESC
FETCH FIRST 5 ROWS ONLY;
```

★ **시험 포인트:** Top N 은 정렬이 먼저입니다.

오답 방지: 정렬 없이 FETCH FIRST 만 쓰면 상위의 의미가 없습니다.

[ROWNUM 방식]

정의: Oracle 전통 방식으로 상위 N 건을 제한하는 방법이다.

핵심 설명: ROWNUM 은 ORDER BY 보다 먼저 붙을 수 있으므로, 정렬 후 상위 N 건을 구하려면 인라인 뷰로 먼저 정렬해야 합니다.

HR 예시 쿼리:

```
-- 올바른 방식: 먼저 정렬하고 바깥에서 ROWNUM 제한
SELECT *
FROM (
    SELECT employee_id, last_name, salary
    FROM HR.EMPLOYEES
    ORDER BY salary DESC
)
WHERE ROWNUM <= 5;
```

★ **시험 포인트:** Oracle Top N 의 대표 함정은 ROWNUM 과 ORDER BY 의 처리 순서입니다.

오답 방지: WHERE ROWNUM <= 5 를 먼저 적용한 뒤 ORDER BY 하면 전체 상위 5 명이 아닐 수 있습니다.

[그룹별 Top N]

정의: 각 그룹 안에서 상위 N 건을 조회하는 방식이다.

핵심 설명: 부서별 급여 상위 3 명처럼 그룹별 Top N 은 단순 FETCH FIRST 로 해결되지 않습니다.

ROW_NUMBER 또는 RANK 를 사용해 그룹별 순위를 만든 뒤 필터링합니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, department_id, salary, rn
FROM (
  SELECT e.*,
         ROW_NUMBER() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC, employee_id
         ) AS rn
  FROM HR.EMPLOYEES e
)
WHERE rn <= 3
ORDER BY department_id, rn;
```

★ **시험 포인트:** 그룹별 Top N 은 윈도우 함수가 필요합니다.

오답 방지: 동점자를 모두 포함하려면 ROW_NUMBER 보다 RANK 또는 DENSE_RANK 를 고려합니다.

13 절 계층형 질의와 셀프 조인

[계층형 데이터]

정의: 부모-자식 관계가 반복되는 트리 구조 데이터다.

핵심 설명: 조직도처럼 한 사원이 다른 사원을 관리하는 구조는 EMPLOYEES 테이블 안의 EMPLOYEE_ID 와 MANAGER_ID 로 표현됩니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, manager_id
FROM HR.EMPLOYEES
WHERE manager_id IS NULL
   OR manager_id = 100
ORDER BY manager_id, employee_id;
```

★ **시험 포인트:** 계층형 데이터는 셀프 조인 또는 Oracle 계층형 질의로 조회합니다.

[셀프 조인]

정의: 같은 테이블을 별칭을 달리하여 자기 자신과 조인하는 것이다.

핵심 설명: 한 단계 관리자 이름을 붙일 때 유용합니다. 같은 EMPLOYEES 테이블을 사원 E 와 관리자 M 으로 나누어 사용합니다.

HR 예시 쿼리:

```
SELECT e.employee_id,  
       e.last_name AS employee_name,  
       m.employee_id AS manager_id,  
       m.last_name AS manager_name  
FROM HR.EMPLOYEES e  
LEFT JOIN HR.EMPLOYEES m  
      ON e.manager_id = m.employee_id  
ORDER BY e.employee_id;
```

★ **시험 포인트:** SELF JOIN 은 별칭이 필수입니다.

[Oracle 계층형 질의]

정의: START WITH, CONNECT BY, LEVEL 을 사용해 계층 구조를 조회하는 Oracle 문법이다.

핵심 설명: START WITH 는 시작 행, CONNECT BY 는 부모-자식 연결 조건, LEVEL 은 계층 깊이를 의미합니다. PRIOR 가 어느 쪽에 붙는지에 따라 탐색 방향을 해석합니다.

HR 예시 쿼리:

```
SELECT LEVEL AS lv,  
       employee_id,  
       LPAD(' ', (LEVEL-1)*2) || last_name AS org_name,  
       manager_id  
FROM HR.EMPLOYEES  
START WITH manager_id IS NULL  
CONNECT BY PRIOR employee_id = manager_id  
ORDER SIBLINGS BY last_name;
```

★ **시험 포인트:** START WITH, CONNECT BY PRIOR, LEVEL 은 Oracle 계층형 질의 핵심입니다.

오답 방지: PRIOR employee_id = manager_id 는 부모의 employee_id 가 자식의 manager_id 와 같다는 뜻입니다.

[SYS_CONNECT_BY_PATH]

정의: 계층 경로를 문자열로 보여주는 Oracle 함수다.

핵심 설명: 루트부터 현재 행까지의 경로를 확인할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id,  
       last_name,  
       SYS_CONNECT_BY_PATH(last_name, ' > ') AS path  
FROM HR.EMPLOYEES  
START WITH manager_id IS NULL  
CONNECT BY PRIOR employee_id = manager_id  
FETCH FIRST 20 ROWS ONLY;
```

★ **시험 포인트:** 계층 경로 확인용 보강 함수입니다.

14 절 PIVOT, UNPIVOT

[PIVOT]

정의: 행 데이터를 열 방향으로 회전해 집계 결과를 만드는 방식이다.

핵심 설명: 세로로 쌓인 분류값을 가로 컬럼으로 펼쳐 보고서 형태를 만들 때 사용합니다.

HR 예시 쿼리:

```
SELECT *
FROM (
    SELECT department_id, job_id, employee_id
    FROM HR.EMPLOYEES
    WHERE job_id IN ('IT_PROG', 'SA_REP', 'ST_CLERK')
)
PIVOT (
    COUNT(employee_id) AS cnt
    FOR job_id IN (
        'IT_PROG' AS it_prog,
        'SA_REP' AS sa_rep,
        'ST_CLERK' AS st_clerk
    )
)
ORDER BY department_id;
```

★ **시험 포인트:** PIVOT 은 행을 열로 바꿉니다.

오답 방지: PIVOT 안에서 어떤 컬럼을 집계하고 어떤 값이 컬럼으로 펼쳐지는지 확인해야 합니다.

[조건부 집계]

정의: CASE 와 집계 함수를 이용해 PIVOT 효과를 만드는 방식이다.

핵심 설명: PIVOT 문법이 익숙하지 않아도 SUM(CASE WHEN ...) 구조를 이해하면 결과 예측이 쉬워집니다.

HR 예시 쿼리:

```
SELECT department_id,
    SUM(CASE WHEN job_id = 'IT_PROG' THEN 1 ELSE 0 END) AS it_prog_cnt,
    SUM(CASE WHEN job_id = 'SA_REP' THEN 1 ELSE 0 END) AS sa_rep_cnt,
    SUM(CASE WHEN job_id = 'ST_CLERK' THEN 1 ELSE 0 END) AS st_clerk_cnt
FROM HR.EMPLOYEES
```

```
GROUP BY department_id
```

```
ORDER BY department_id;
```

★ **시험 포인트:** CASE 가 어떤 행을 어떤 컬럼 집계에 넣는지 확인해야 합니다.

[UNPIVOT]

정의: 여러 컬럼을 행 방향으로 풀어내는 방식이다.

핵심 설명: 가로로 펼쳐진 컬럼을 세로 행 구조로 바꿉니다.

HR 예시 쿼리:

```
SELECT employee_id, pay_type, amount
FROM (
    SELECT employee_id,
           salary,
           NVL(salary * commission_pct, 0) AS commission_amount
    FROM HR.EMPLOYEES
    WHERE employee_id IN (145, 146)
)
UNPIVOT (
    amount FOR pay_type IN (
        salary AS 'SALARY',
        commission_amount AS 'COMMISSION'
    )
);
```

★ **시험 포인트:** UNPIVOT 은 열을 행으로 바꿉니다.

15 절 정규 표현식

[정규 표현식]

정의: 문자열의 복잡한 패턴을 표현하는 규칙이다.

핵심 설명: LIKE 보다 복잡한 문자 조건을 표현할 수 있습니다. 이메일, 전화번호, 코드 패턴 검사에 사용합니다.

HR 예시 쿼리:

```
SELECT employee_id, email
FROM HR.EMPLOYEES
WHERE REGEXP_LIKE(email, '^[A-Z]+$')
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** LIKE 의 %, _와 정규식의 *, +, .은 의미가 다릅니다.

[REGEXP_LIKE] 출제 빈도 높음

정의: 문자열이 정규 표현식 패턴과 일치하는지 검사한다.

핵심 설명: ^는 시작, \$는 끝을 의미합니다.

HR 예시 쿼리:

```
SELECT employee_id, phone_number
FROM HR.EMPLOYEES
WHERE REGEXP_LIKE(phone_number, '^515')
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** ^는 문자열 시작입니다.

[REGEXP_REPLACE] 출제 빈도 높음

정의: 패턴에 맞는 문자열을 다른 문자열로 바꾼다.

핵심 설명: 전화번호에서 숫자가 아닌 문자를 제거하는 식으로 사용할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, phone_number,
       REGEXP_REPLACE(phone_number, '[^0-9]', '') AS only_digits
FROM HR.EMPLOYEES
WHERE phone_number IS NOT NULL
```

FETCH FIRST 10 ROWS ONLY;

★ **시험 포인트:** [^0-9]는 숫자가 아닌 문자입니다.

[REGEXP_SUBSTR] 출제 빈도 높음

정의: 패턴에 맞는 부분 문자열을 추출한다.

핵심 설명: 문자열 안에서 원하는 패턴만 뽑습니다.

HR 예시 쿼리:

```
SELECT employee_id, phone_number,  
       REGEXP_SUBSTR(phone_number, '[0-9]+') AS first_number_group  
FROM HR.EMPLOYEES  
WHERE phone_number IS NOT NULL  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** +는 앞 패턴이 1 회 이상 반복된다는 뜻입니다.

[REGEXP_INSTR] 출제 빈도 높음

정의: 패턴이 나타나는 위치를 반환한다.

핵심 설명: INSTR 의 정규 표현식 버전으로 이해할 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, email,  
       REGEXP_INSTR(email, 'A') AS pos_a  
FROM HR.EMPLOYEES  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 없으면 0 을 반환할 수 있습니다.

[REGEXP_COUNT] 출제 빈도 높음

정의: 패턴이 몇 번 등장하는지 센다.

핵심 설명: 문자열 내 특정 패턴 반복 횟수를 셀 때 사용합니다.

HR 예시 쿼리:


```
SELECT employee_id, phone_number,  
       REGEXP_COUNT(phone_number, '[0-9]') AS digit_count  
FROM HR.EMPLOYEES  
WHERE phone_number IS NOT NULL  
FETCH FIRST 10 ROWS ONLY;
```

★ **시험 포인트:** 패턴 반복 횟수 계산 함수입니다.

정규 표현식 기호 빠른 암기

- ^: 문자열 시작
- \$: 문자열 끝
- .: 임의의 한 문자
- *: 앞 패턴 0 회 이상 반복
- +: 앞 패턴 1 회 이상 반복
- []: 후보 문자 집합
- [^]: 후보 문자 집합 제외
- |: OR
- {m,n}: m 회 이상 n 회 이하 반복

16 절 DML

[DML]

정의: 테이블의 데이터를 조회, 입력, 수정, 삭제하는 명령어다.

핵심 설명: SELECT, INSERT, UPDATE, DELETE, MERGE 가 대표적입니다. 변경된 데이터는 트랜잭션 안에서 COMMIT 또는 ROLLBACK 대상이 될 수 있습니다.

HR 예시 쿼리:

```
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE department_id = 60;
```

★ **시험 포인트:** DELETE 는 DML 이고 TRUNCATE 는 DDL 성격입니다.

[INSERT]

정의: 테이블에 새 행을 입력하는 명령어다.

핵심 설명: 컬럼 목록을 명시하면 컬럼 순서 오류를 줄일 수 있습니다. HR.DEPARTMENTS 는 실습 예시로 비교적 단순합니다.

HR 예시 쿼리:

```
INSERT INTO HR.DEPARTMENTS
    (department_id, department_name, manager_id, location_id)
VALUES
    (999, 'TEMP_DEPT', NULL, 1700);
```

★ **시험 포인트:** 컬럼 목록을 생략하면 테이블 컬럼 순서대로 모든 값을 맞춰야 합니다.

오답 방지: 실제 HR 샘플에 실행하면 데이터가 추가됩니다. 실습 후 ROLLBACK 또는 DELETE 를 고려하세요.

[INSERT SELECT]

정의: SELECT 결과를 다른 테이블에 입력하는 방식이다.

핵심 설명: 테이블 구조가 맞으면 조회 결과를 여러 행으로 입력할 수 있습니다.

HR 예시 쿼리:

```
CREATE TABLE emp_60 AS
SELECT employee_id, last_name, salary
```

```
FROM HR.EMPLOYEES

WHERE 1 = 0;

INSERT INTO emp_60 (employee_id, last_name, salary)
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE department_id = 60;
```

★ **시험 포인트:** INSERT SELECT 는 VALUES 없이 SELECT 결과를 입력합니다.

[UPDATE]

정의: 기존 행의 컬럼 값을 수정하는 명령어다.

핵심 설명: SET 으로 바꿀 값을 지정하고 WHERE 로 수정 대상을 제한합니다. WHERE 를 빠뜨리면 모든 행이 수정될 수 있습니다.

HR 예시 쿼리:

```
UPDATE HR.DEPARTMENTS
SET department_name = 'TEMP_DEPT_RENAMED'
WHERE department_id = 999;
```

★ **시험 포인트:** UPDATE 에서 WHERE 누락은 전체 행 수정으로 이어집니다.

[DELETE]

정의: 조건에 맞는 행을 삭제하는 명령어다.

핵심 설명: DELETE 는 행 단위 삭제 명령입니다. 조건이 없으면 모든 행을 삭제하지만 테이블 구조는 유지됩니다.

HR 예시 쿼리:

```
DELETE FROM HR.DEPARTMENTS
WHERE department_id = 999;
```

★ **시험 포인트:** DELETE 는 DML 이므로 트랜잭션 처리 대상이 될 수 있습니다.

오답 방지: DELETE 는 테이블 구조를 삭제하지 않습니다.

[MERGE]

정의: 조건에 따라 UPDATE 또는 INSERT 를 수행하는 명령어다.

핵심 설명: 대상 테이블에 행이 있으면 UPDATE, 없으면 INSERT 하는 upsert 작업을 수행합니다.

HR 예시 쿼리:

```
MERGE INTO HR.DEPARTMENTS d
USING (
    SELECT 999 AS department_id,
           'TEMP_DEPT' AS department_name,
           1700 AS location_id
    FROM DUAL
) s
ON (d.department_id = s.department_id)
WHEN MATCHED THEN
    UPDATE SET d.department_name = s.department_name
WHEN NOT MATCHED THEN
    INSERT (department_id, department_name, location_id)
    VALUES (s.department_id, s.department_name, s.location_id);
```

★ **시험 포인트:** MERGE 는 INSERT 와 UPDATE 를 결합한 성격입니다.

17 절 TCL

[TCL]

정의: DML 작업 결과를 확정하거나 취소하는 트랜잭션 제어 명령어다.

핵심 설명: COMMIT, ROLLBACK, SAVEPOINT 가 대표적입니다. 여러 DML 을 하나의 작업 단위로 묶을 때 중요합니다.

HR 예시 쿼리:

```
INSERT INTO HR.DEPARTMENTS (department_id, department_name, location_id)
```

```
VALUES (998, 'TX_TEST', 1700);
```

```
ROLLBACK;
```

★ **시험 포인트:** TCL 은 DML 결과를 제어합니다.

[COMMIT]

정의: 트랜잭션 변경 내용을 확정하는 명령이다.

핵심 설명: COMMIT 후에는 변경 내용이 데이터베이스에 확정됩니다. 일반적으로 COMMIT 된 변경은 ROLLBACK 으로 되돌릴 수 없습니다.

HR 예시 쿼리:

```
INSERT INTO HR.DEPARTMENTS (department_id, department_name, location_id)
```

```
VALUES (997, 'COMMIT_TEST', 1700);
```

```
COMMIT;
```

★ **시험 포인트:** COMMIT 은 변경 사항 확정입니다.

오답 방지: 학습용 예제 실행 시 COMMIT 하면 실제 데이터가 남을 수 있습니다.

[ROLLBACK]

정의: 아직 확정되지 않은 변경 내용을 취소하는 명령이다.

핵심 설명: 마지막 COMMIT 이후의 변경 내용을 취소합니다.

HR 예시 쿼리:

```
UPDATE HR.DEPARTMENTS  
SET department_name = 'WRONG_NAME'  
WHERE department_id = 60;  
  
ROLLBACK;
```

★ **시험 포인트:** COMMIT 전 DML 은 ROLLBACK 할 수 있습니다.

[SAVEPOINT]

정의: 트랜잭션 중간에 되돌아갈 지점을 지정하는 명령이다.

핵심 설명: 전체 ROLLBACK 이 아니라 특정 지점까지만 되돌릴 수 있습니다.

HR 예시 쿼리:

```
INSERT INTO HR.DEPARTMENTS (department_id, department_name, location_id)  
VALUES (996, 'SAVEPOINT_TEST', 1700);  
  
SAVEPOINT s1;  
  
UPDATE HR.DEPARTMENTS  
SET department_name = 'SAVEPOINT_TEST_2'  
WHERE department_id = 996;  
  
ROLLBACK TO s1;  
ROLLBACK;
```

★ **시험 포인트:** SAVEPOINT 는 COMMIT 이 아니라 중간 되돌림 지점입니다.

[트랜잭션 ACID]

정의: 원자성, 일관성, 고립성, 지속성이다.

핵심 설명: 원자성은 전부 성공 또는 전부 실패, 일관성은 처리 전후 데이터 규칙 유지, 고립성은 동시 실행 간섭 최소화, 지속성은 완료 결과 보존입니다.

HR 예시 쿼리:

-- 개념 예시: 부서 생성과 관련 이력 입력이 하나의 업무라면

-- 둘 다 성공하면 COMMIT, 하나라도 실패하면 ROLLBACK 해야 원자성이 유지됩니다.

SAVEPOINT before_work;

-- INSERT INTO ...

-- INSERT INTO ...

-- 문제가 생기면 ROLLBACK TO before_work;

★ **시험 포인트:** ACID 의 네 단어와 의미를 구분합니다.

18 절 DDL

[DDL]

정의: 테이블, 뷰, 인덱스 같은 데이터베이스 객체의 구조를 정의하는 명령어다.

핵심 설명: CREATE, ALTER, DROP, TRUNCATE, RENAME 이 대표적입니다. DDL 은 일반적으로 자동 COMMIT 성격이 강합니다.

HR 예시 쿼리:

```
CREATE TABLE emp_temp AS
SELECT employee_id, last_name, salary
FROM HR.EMPLOYEES
WHERE 1 = 0;
```

★ **시험 포인트:** DDL 은 구조 정의, DML 은 데이터 조작입니다.

[CREATE]

정의: 새로운 데이터베이스 객체를 생성한다.

핵심 설명: CREATE TABLE 은 테이블과 컬럼, 데이터 타입, 제약조건을 정의합니다.

HR 예시 쿼리:

```
CREATE TABLE dept_temp (
    department_id NUMBER(4) PRIMARY KEY,
    department_name VARCHAR2(30) NOT NULL,
    location_id NUMBER(4) DEFAULT 1700
);
```

★ **시험 포인트:** CREATE 는 데이터를 입력하는 것이 아니라 구조를 만드는 명령입니다.

[ALTER]

정의: 기존 객체 구조를 변경한다.

핵심 설명: 컬럼 추가, 컬럼 변경, 컬럼 삭제, 제약조건 추가/삭제 등에 사용됩니다.

HR 예시 쿼리:

```
ALTER TABLE dept_temp
ADD created_at DATE DEFAULT SYSDATE;
```


★ **시험 포인트:** ALTER 는 데이터 행 수정이 아니라 테이블 구조 변경입니다.

[DROP]

정의: 객체 자체를 삭제한다.

핵심 설명: DROP TABLE 은 테이블 구조와 데이터를 모두 삭제합니다.

HR 예시 쿼리:

```
DROP TABLE dept_temp;
```

★ **시험 포인트:** DROP 은 테이블의 행만 삭제하는 DELETE 와 다릅니다.

[TRUNCATE]

정의: 테이블 구조는 유지하고 모든 행을 빠르게 삭제한다.

핵심 설명: TRUNCATE 는 WHERE 조건 없이 전체 데이터를 제거합니다. DDL 성격이 강하며 일반적으로 롤백이 어렵거나 자동 커밋될 수 있습니다.

HR 예시 쿼리:

```
TRUNCATE TABLE emp_temp;
```

★ **시험 포인트:** TRUNCATE 는 전체 행 삭제이며 WHERE 조건을 사용할 수 없습니다.

[RENAME]

정의: 객체 이름을 변경한다.

핵심 설명: 테이블 같은 객체의 이름을 바꿀 때 사용합니다.

HR 예시 쿼리:

```
RENAME emp_temp TO emp_temp_backup;
```

★ **시험 포인트:** 이름만 바뀌지만 참조 관계와 권한 영향을 확인해야 합니다.

[VIEW]

정의: SELECT 결과를 가상의 테이블처럼 정의한 객체다.

핵심 설명: 복잡한 SELECT 를 저장해 재사용할 수 있습니다.

HR 예시 쿼리:

```
CREATE OR REPLACE VIEW emp_dept_v AS
SELECT e.employee_id, e.last_name, d.department_name
FROM HR.EMPLOYEES e
LEFT JOIN HR.DEPARTMENTS d
  ON e.department_id = d.department_id;
```

★ **시험 포인트:** VIEW 는 저장된 SELECT 정의로 이해합니다.

[INDEX]

정의: 검색 성능 향상을 위해 컬럼에 생성하는 객체다.

핵심 설명: WHERE, JOIN, ORDER BY 에 자주 쓰는 컬럼에 인덱스를 만들 수 있습니다.

HR 예시 쿼리:

```
CREATE INDEX emp_temp_salary_ix
ON emp_temp_backup (salary);
```

★ **시험 포인트:** 인덱스는 조회 성능에 도움을 줄 수 있지만 무조건 좋은 것은 아닙니다.

오답 방지: INSERT/UPDATE/DELETE 시 인덱스 유지 비용이 생길 수 있습니다.

[제약조건]

정의: 데이터 무결성을 지키기 위해 컬럼이나 테이블에 설정하는 규칙이다.

핵심 설명: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT 가 대표적입니다.

HR 예시 쿼리:

```
CREATE TABLE dept_temp (
  department_id  NUMBER(4) CONSTRAINT dept_temp_pk PRIMARY KEY,
  department_name VARCHAR2(30) NOT NULL,
  location_id    NUMBER(4) DEFAULT 1700,
  CONSTRAINT dept_temp_location_ck CHECK (location_id > 0)
);
```

★ **시험 포인트:** PK 는 중복 불가와 NULL 불가를 모두 가집니다.

오답 방지: UNIQUE 는 중복을 막지만 NULL 처리 방식은 DBMS 별로 차이가 있을 수 있습니다.

[DELETE, TRUNCATE, DROP 비교]

정의: DELETE 는 행 삭제, TRUNCATE 는 전체 행 제거, DROP 은 객체 삭제다.

핵심 설명: DELETE 는 DML 로 조건 삭제가 가능하고 트랜잭션 처리 대상입니다. TRUNCATE 는 DDL 성격으로 전체 행을 빠르게 제거합니다. DROP 은 테이블 자체를 없앱니다.

HR 예시 쿼리:

```
DELETE FROM emp_temp_backup WHERE salary < 3000; -- 조건 삭제 가능
TRUNCATE TABLE emp_temp_backup;                -- 전체 행 삭제, 구조 유지
DROP TABLE emp_temp_backup;                      -- 테이블 객체 삭제
```

★ **시험 포인트:** DELETE/TRUNCATE/DROP 차이는 반복 출제됩니다.

19 절 DCL

[DCL]

정의: 데이터베이스 접근 권한을 부여하거나 회수하는 명령어다.

핵심 설명: GRANT 와 REVOKE 가 대표적입니다. 사용자가 어떤 객체에 어떤 작업을 할 수 있는지 제어합니다.

HR 예시 쿼리:

```
GRANT SELECT ON HR.EMPLOYEES TO user1;
```

★ **시험 포인트:** DCL 은 데이터 조회 결과가 아니라 권한 제어와 관련됩니다.

[GRANT]

정의: 사용자나 역할에 권한을 부여하는 명령어다.

핵심 설명: SELECT, INSERT, UPDATE, DELETE 같은 객체 권한을 사용자에게 부여할 수 있습니다. WITH GRANT OPTION 이 있으면 받은 권한을 다른 사용자에게 다시 부여할 수 있습니다.

HR 예시 쿼리:

```
GRANT SELECT, INSERT ON HR.DEPARTMENTS TO user1;
```

```
GRANT SELECT ON HR.EMPLOYEES TO user1 WITH GRANT OPTION;
```

★ **시험 포인트:** WITH GRANT OPTION 은 권한 재부여 가능 여부와 연결됩니다.

[REVOKE]

정의: 부여된 권한을 회수하는 명령어다.

핵심 설명: 사용자가 가진 권한을 취소합니다. 권한 회수 범위와 연쇄 회수 여부는 DBMS 정책에 따라 달라질 수 있습니다.

HR 예시 쿼리:

```
REVOKE INSERT ON HR.DEPARTMENTS FROM user1;
```

```
REVOKE SELECT ON HR.EMPLOYEES FROM user1;
```

★ **시험 포인트:** REVOKE 는 실행 취소 ROLLBACK 이 아니라 권한 회수 명령입니다.

[ROLE]

정의: 여러 권한을 묶어 관리하는 권한 집합이다.

핵심 설명: 여러 사용자에게 동일한 권한 묶음을 쉽게 부여하기 위한 도구입니다.

HR 예시 쿼리:

```
CREATE ROLE hr_read_role;  
  
GRANT SELECT ON HR.EMPLOYEES TO hr_read_role;  
  
GRANT SELECT ON HR.DEPARTMENTS TO hr_read_role;  
  
GRANT hr_read_role TO user1;
```

★ **시험 포인트:** ROLE 은 권한의 묶음입니다.

20 절 시험 직전 압축 정리

[빨간색 함수 최우선 암기]

NULL 처리: NVL, NVL2, COALESCE, NULLIF

조건/변환: CASE, DECODE, TO_CHAR, TO_DATE, TO_NUMBER

집계/그룹: COUNT(*), COUNT(컬럼), SUM, AVG, MIN, MAX, ROLLUP, CUBE, GROUPING SETS, GROUPING

윈도우: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, ROWS/RANGE

정규표현식: REGEXP_LIKE, REGEXP_REPLACE, REGEXP_SUBSTR, REGEXP_INSTR, REGEXP_COUNT

SELECT 실행 순서

- 논리적 처리 순서: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> FETCH
- WHERE 는 행 조건, HAVING 은 그룹 조건입니다.
- ORDER BY 에서는 SELECT 별칭 사용이 가능합니다.

Oracle 함수 핵심

- 문자: UPPER, LOWER, INITCAP, SUBSTR, LENGTH, INSTR, LPAD/RPAD, TRIM, REPLACE, TRANSLATE
- 숫자: ROUND, TRUNC, CEIL, FLOOR, MOD, ABS, SIGN
- 날짜: SYSDATE, ADD_MONTHS, MONTHS_BETWEEN, LAST_DAY, EXTRACT, TRUNC(date)
- 변환: TO_CHAR, TO_DATE, TO_NUMBER, CAST
- NULL: NVL, NVL2, COALESCE, NULLIF
- 조건: CASE, DECODE

집계와 그룹 함수 핵심

- COUNT(*)는 행 수, COUNT(컬럼)은 NULL 제외 값 수입니다.
- SUM, AVG, MIN, MAX 는 NULL 을 제외하고 계산합니다.
- ROLLUP 은 계층 소계, CUBE 는 모든 조합 소계, GROUPING SETS 는 필요한 조합만 지정합니다.
- GROUPING 은 소계 NULL 과 실제 NULL 을 구분합니다.

JOIN 핵심

- INNER JOIN 은 매칭 행만 반환합니다.
- LEFT OUTER JOIN 은 왼쪽 테이블 행을 보존합니다.
- CROSS JOIN 은 모든 조합을 반환합니다.
- NATURAL JOIN 은 같은 이름 컬럼을 자동 조건으로 사용하므로 함정이 될 수 있습니다.
- Oracle 구식 외부 조인 (+)는 NULL 로 채워질 수 있는 선택 테이블 쪽에 붙습니다.

서브쿼리 핵심

- 스칼라 서브쿼리는 한 행 한 컬럼입니다.
- IN 은 목록 포함, EXISTS 는 행 존재 여부입니다.
- ANY 는 하나라도 만족, ALL 은 모두 만족입니다.
- NOT IN + NULL 은 결과가 비어 보일 수 있어 주의합니다.

윈도우 함수 핵심

- 윈도우 함수는 행을 줄이지 않습니다.
- ROW_NUMBER 는 고유 번호, RANK 는 순위 건너뛰, DENSE_RANK 는 순위 건너뛰지 않음입니다.
- LAG 는 이전 행, LEAD 는 다음 행입니다.
- LAST_VALUE 는 프레임 범위를 명시하지 않으면 함정이 될 수 있습니다.

관리 구문 핵심

- DML: SELECT, INSERT, UPDATE, DELETE, MERGE
- TCL: COMMIT, ROLLBACK, SAVEPOINT
- DDL: CREATE, ALTER, DROP, TRUNCATE, RENAME
- DCL: GRANT, REVOKE
- DELETE 는 DML, TRUNCATE 와 DROP 은 DDL 성격입니다.